PROGRAMAÇÃO DINÂMICA

Recordando Divisão e Conquista

Coeficiente Binomial

$$\binom{n}{k} = \begin{cases} 1 & \text{se } k = 0 \text{ ou } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{se } 0 < k < n \\ 0 & \text{caso contrário} \end{cases}$$

Recordando Divisão e Conquista

Coeficiente Binomial

$$\binom{n}{k} = \begin{cases} 1 & \text{se } k = 0 \text{ ou } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{se } 0 < k < n \\ 0 & \text{caso contrário} \end{cases}$$

Função coeficiente_bin (*n*,*k*)

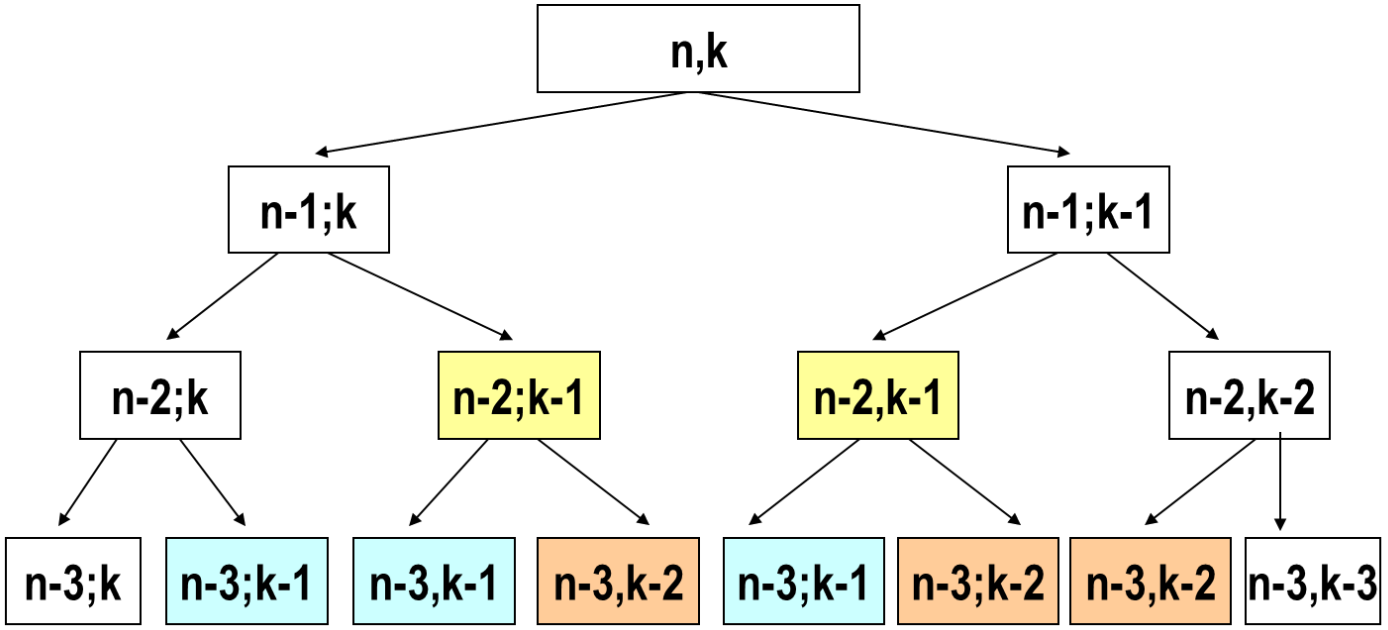
se (*k*=0) ou (*k*=*n*) então retorne (1);

senão retorne (coeficiente_bin (*n*-1,*k*-1) + coeficiente_bin (*n*-1,*k*));

$$T_n = \begin{cases} 1, \text{ se } k = 0 \text{ ou } k = n \\ 2T_{n-1}, \text{ caso contrário} \end{cases}$$

Recordando Divisão e Conquista

Coeficiente Binomial



Recordando Divisão e Conquista

Coeficiente Binomial

$$T_n = \begin{cases} 1, \text{ se } k = 0 \text{ ou } k = n \\ 2T_{n-1}, \text{ caso contrário} \end{cases}$$

$x^2 - 2x = 0$

Eq característica: x-2 = 0
Raiz da eq. Carac.: x = 2

$T_n = c_1 2^n$

$T_0 = c_1 2^0 = 1$

$c_1 = 1$

$T_n = 2^n$

Estratégia *Bottom-Up*

Coeficiente Binomial

$$S_0 = \left\{ \binom{0}{0} \right\}$$

$$S_1 = \left\{ \binom{1}{0}, \binom{1}{1} \right\}$$

$$S_2 = \left\{ \binom{2}{0}, \binom{2}{1}, \binom{2}{2} \right\}$$

$$\vdots$$

$$S_n = \left\{ \binom{n}{0}, \binom{n}{1}, \binom{n}{2}, \ldots, \binom{n}{n} \right\}$$

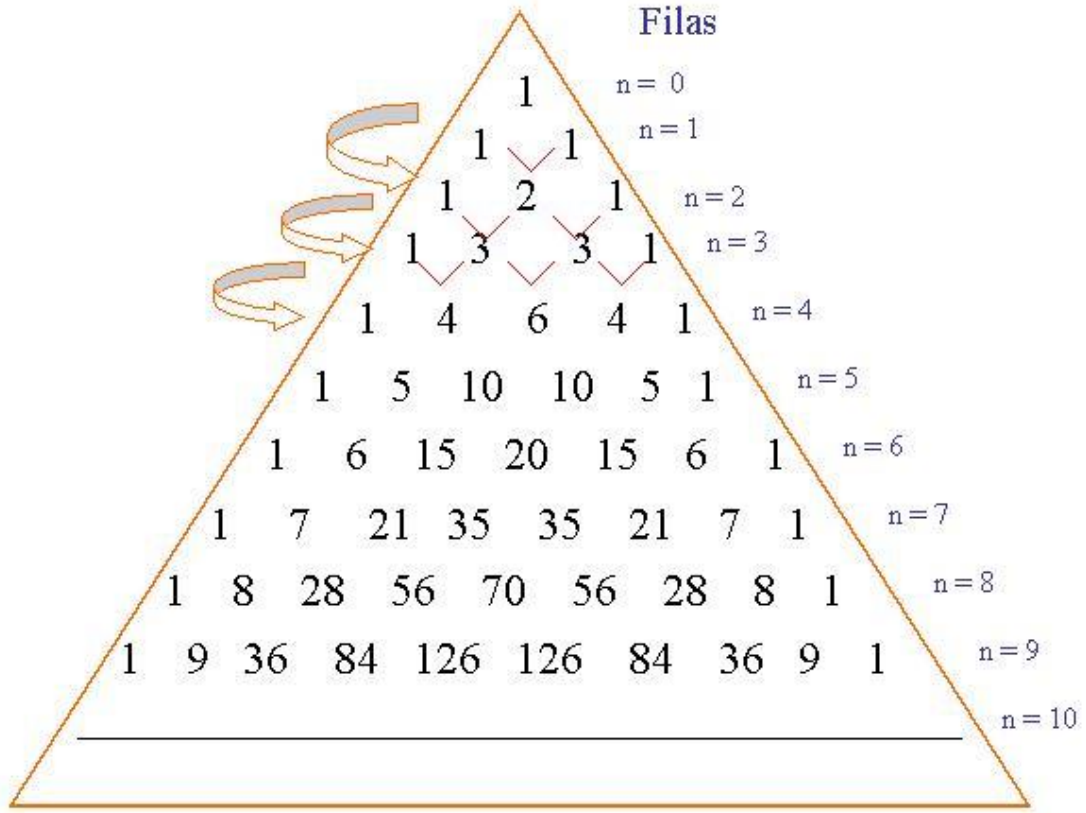
S_{i+1} pode ser calculado a partir dos resultados de S_i.

Triângulo de Pascal

Triângulo de Pascal

Coeficiente Binomial

$$C\binom{n}{k} = C\binom{n-1}{k-1} + C\binom{n-1}{k}$$



Coeficiente Binomial

```
Função coeficiente_bin (n,m)
    int b[n+1]
    b[0] ← 1
    para i ← 1 até n
    {   b[i] ← 1
        para j ← i-1 até 0 (passo -1)
        b[j] ← b[j] + b[j-1]
    }
    retorne(b[m])
```

Inicia pelos menores problemas

A cada iteração, guarda as soluções anteriores

Programação Dinâmica

- A programação dinâmica nos fornece uma maneira de projetar algoritmos que:
  - *Sistematicamente explora todas as possibilidades (corretude);*
  - *Armazena resultados a fim de evitar computação redundante (eficiência);*
  - *Define a solução para o problema em termos da solução de problemas menores.*

Programação Dinâmica

- Essa técnica evita a computação dos mesmos subproblemas repetidamente pois:
  - *Começa a resolver o problema pelos subproblemas de menor tamanho (estratégia bottom-up);*
  - *A medida que são computados, armazena as respostas;*
  - *Quando o mesmo subproblema precisa ser resolvido novamente, apenas consulta o valor armazenado.*

Programação Dinâmica

- Pode ser descrito em uma lista de quatro tarefas:
  - *Caracterizar a estrutura de uma solução ótima;*
  - *Definir recursivamente o valor de uma solução ótima;*
  - *Resolver o problema pelos subproblemas de menor tamanho (estratégia bottom-up); À medida que são computados, armazena as respostas; Quando o mesmo subproblema precisa ser resolvido novamente, apenas consulta o valor armazenado.*
  - *Construir a solução ótima a partir da informação computada.*

Programação Dinâmica

- Aplicável a problemas que possuam as propriedades:
  - *Subestrutura ótima*
  - *Superposição de problemas*

Subestrutura Ótima

- O problema pode ser dividido sucessivamente, e a combinação das soluções ótimas dos subproblemas corresponde à solução ótima do problema original.

Exemplo: Caminho mais curto em grafos

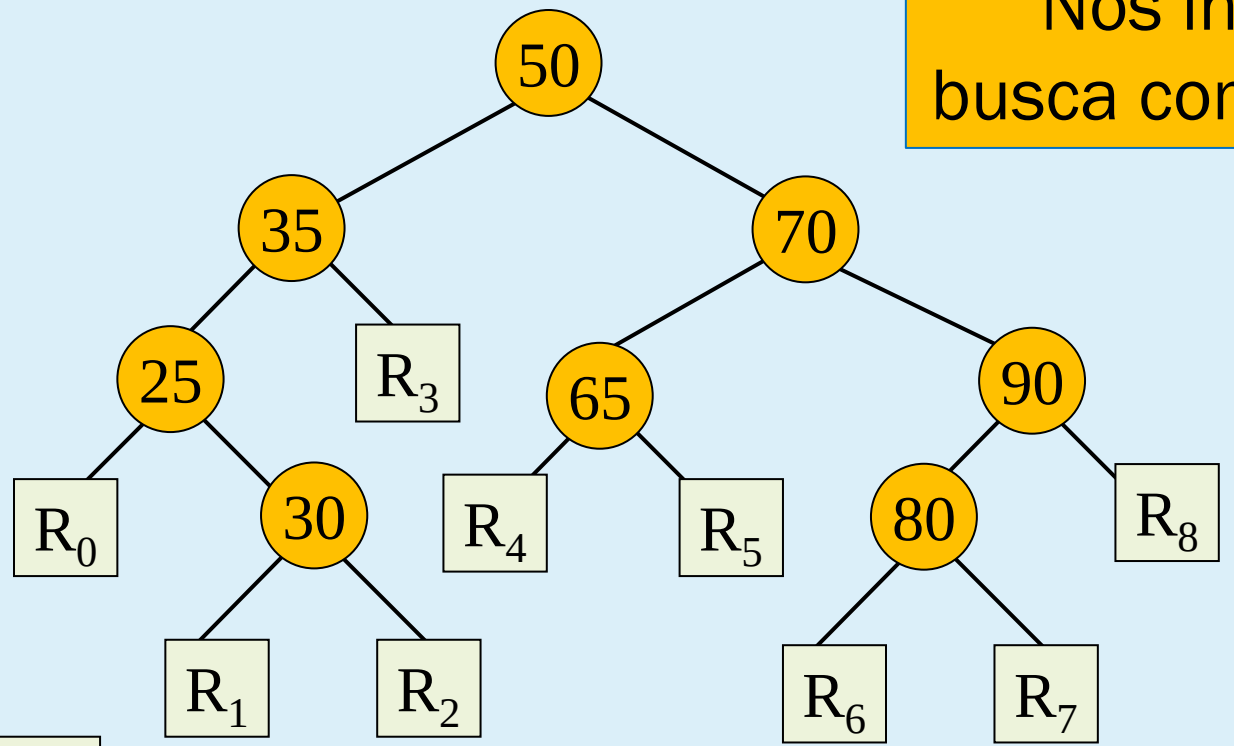
Superposição de Problemas

- O espaço de subproblemas é pequeno, e eles se repetem durante a solução do problema original.
- Podemos pensar que um algoritmo recursivo para o problema de Programação Dinâmica, resolveria os mesmos subproblemas repetidas vezes, sem gerar novos subproblemas.
- A abordagem "*dividir e conquistar*" quase sempre gera subproblemas absolutamente novos.

Superposição de Problemas

- O espaço de subproblemas é pequeno, e eles se repetem durante a solução do problema original.
- Podemos pensar que um algoritmo recursivo para o problema de Programação Dinâmica, resolveria os mesmos subproblemas repetidas vezes, sem gerar novos subproblemas.
- A abordagem "*dividir e conquistar*" quase sempre gera subproblemas absolutamente novos.

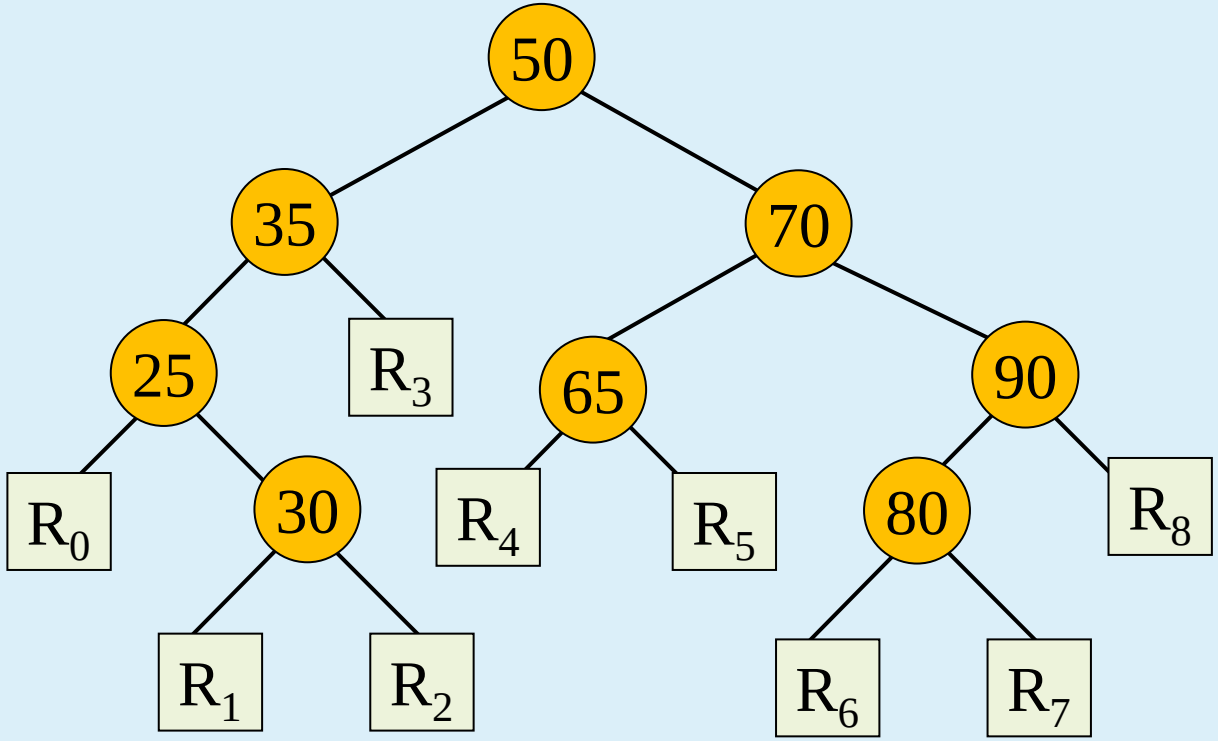
Árvore Binária de Busca



Nós internos
busca com sucesso

Nós externos
busca sem sucesso

Árvore Binária de Busca Ótima



- Equiprobabilidade
- Árvore ótima: Comprimento de caminho interno é mínimo.
- Árvore completa
- Altura da árvore é $O(\log n)$

n é o número de nós na árvore

Árvore Binária de Busca – Frequências de Acesso Diferentes

Considere que cada chave s_i, $i = 1, ..., n$, possui probabilidade de acesso p_i, tal que $p_1, p_2, ..., p_n$, se distribuem de forma qualquer.

Além disso, os nós externos $R_0, ..., R_n$, possuem probabilidades $p_0', p_1', ..., p_n'$. Dadas as probabilidades, deseja-se construir a ***Árvore Ótima*** para efetuar a busca.

Obs. Se $p_1 = p_2 = ... = p_n = p_0' = p_1' = ... = p_n'$, então a *árvore ótima* é a *árvore completa*.

Comprimento de Caminho Interno Ponderado

- Busca com Sucesso

- São necessárias nível(s_k) comparações para encontrar s_k.

- A probabilidade de acesso a s_k é p_k.

- A contribuição deste nó é p_k nível(s_k).

- Para todos os nós internos tem-se: $\sum_{i=1}^{n} p_i nível(s_i)$

Comprimento de Caminho Externo Ponderado

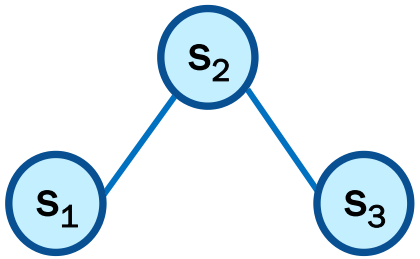
- Busca sem Sucesso

$$\sum_{i=0}^{n} p_i' \left(nível(R_i) - 1 \right)$$

Exemplo

- p_1= 0,4; p_2= 0,1; p_3= 0,1
- p_0'= p_1'= ...= p_3' = 0,1

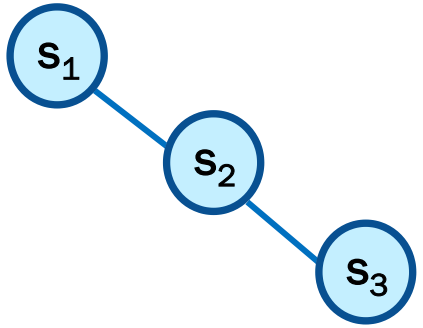
Árvore Completa



c(T) = (0,4)2 + (0,1)1 + (0,1)2 +
(0,1)2 + (0,1)2 + (0,1)2 + (0,1)2

C(T) = 1,9

Árvore Zig-zag



c(T) = (0,4)1 + (0,1)2 + (0,1)3 +
(0,1)1 + (0,1)2 + (0,1)3 + (0,1)3

C(T) = 1,8

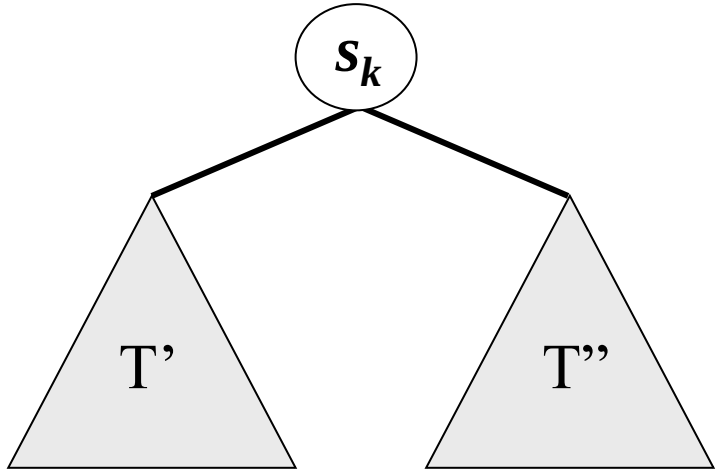
Árvore Binária de Busca Ótima

- Minimizar a expressão ($c(T)$ é o custo da árvore)

$$c(T) = \sum_{i=1}^{n} p_i \, nível(s_i) + \sum_{i=0}^{n} p'_i \left(nível(R_i) - 1 \right)$$

Árvore de Custo Mínimo

- Considere o conjunto de chaves $\{s_1, ..., s_n\}$ s_i, $i = 1, ..., n$.
- Suponha que a raiz s_k da árvore ótima é conhecida.



T' → subárvore binária de busca $\{s_1, ..., s_{k-1}\}$
T'' → subárvore binária de busca $\{s_{k+1}, ..., s_n\}$

Árvore de Custo Mínimo

- Lema. As subárvores de uma árvore binária de busca ótima, são também ótimas.

- **Prova**.

Se isso não ocorresse, então a substituição da subárvore não ótima pela ótima, resultaria em uma diminuição do custo total, o que é uma contradição com o fato da árvore ser ótima. Portanto, T' e T" são árvores ótimas.

Árvore de Custo Mínimo

- Questões:

1. Como determinar s_k?

2. Como construir T' e T"?

- Para resolver 1, a ideia é tentar todas as $O(n)$ alternativas.

- Para resolver 2, usar a recursão.

Árvore de Custo Mínimo

- Seja $c(T)$ o custo da árvore T.

É necessário encontrar uma relação entre os custos $c(T)$, $c(T')$ e $c(T'')$.

Sabe-se que:

$$c(T) = \sum_{i=1}^{n} p_i \, nível(s_i) + \sum_{i=0}^{n} p_i' \left(nível(R_i) - 1 \right)$$

Árvore de Custo Mínimo

$$c(T) = \sum_{i=1}^{n} p_i \, nível(s_i) + \sum_{i=0}^{n} p_i' \left(nível(R_i) - 1 \right)$$

$l_i \longrightarrow$ nível(s_i); $l_i' \longrightarrow$ nível(R_i)

T(i,j) $\longrightarrow$ a árvore de busca ótima para as chaves {s_{i+1}, ..., s_j}, $i \leq j$.

T(i,i) = 0 e T(0,n) = árvore ótima final

$c(i,j) \longrightarrow$ custo da árvore T(i,j)

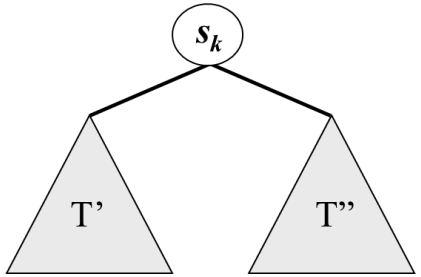
P(i,j) $\longrightarrow$ a soma das probabilidades relativas as chaves {s_{i+1}, ..., s_j}

$$P(i, j) = \sum_{k=i+1}^{j} p_k + \sum_{k=i}^{j} p_k'$$

Árvore de Custo Mínimo

- **Lema**. Se s_k é a raiz de T, então para $i < j$, tem-se:

$$c(i,j) = c(i, k\text{-}1) + c(k,j) + P(i,j)$$



Prova

Nível de s_m em T(i,k-1)

contribuição da raiz

$$c(i,j) = p_k(1) + \underbrace{\sum_{m=i+1}^{k-1} p_m(l_m+1) + \sum_{m=i}^{k-1} p'_m(l'_m-1+1)}_{T(i,\,k\text{-}1)} + \underbrace{\sum_{m=k+1}^{j} p_m(l_m+1) + \sum_{m=k}^{j} p'_m(l'_m-1+1)}_{T(k,\,j)}$$

Árvore de Custo Mínimo

Prova (cont.)

$$c(i,j) = \underbrace{p_k}_{\text{I}} + \underbrace{\sum_{m=i+1}^{k-1} p_m l_m}_{\text{II}} + \underbrace{\sum_{m=i}^{k-1} p'_m \left(l'_m - 1\right)}_{\text{III}} + \underbrace{\sum_{m=k+1}^{j} p_m l_m}_{\text{IV}}$$

$$+ \underbrace{\sum_{m=k}^{j} p'_m \left(l'_m - 1\right)}_{\text{V}} + \underbrace{\sum_{m=i+1}^{k-1} p_m}_{\text{VI}} + \underbrace{\sum_{m=k+1}^{j} p_m}_{\text{VII}} + \underbrace{\sum_{m=i}^{k-1} p'_m}_{\text{VIII}} + \underbrace{\sum_{m=k+1}^{j} p'_m}_{\text{IX}}$$

Árvore de Custo Mínimo

Prova (cont.)

- P(*i*,*j*) = I + VI + VII + VIII + IX
- c(*i*,*k*-1) = II + III
- c(*k*,*j*) = IV + V

$$c(i,j) = \underbrace{p_k}_{\text{I}} + \underbrace{\sum_{m=i+1}^{k-1} p_m l_m}_{\text{II}} + \underbrace{\sum_{m=i}^{k-1} p'_m \left(l'_m - 1\right)}_{\text{III}} + \underbrace{\sum_{m=k+1}^{j} p_m l_m}_{\text{IV}}$$

$$+ \underbrace{\sum_{m=k}^{j} p'_m \left(l'_m - 1\right)}_{\text{V}} + \underbrace{\sum_{m=i+1}^{k-1} p_m}_{\text{VI}} + \underbrace{\sum_{m=k+1}^{j} p_m}_{\text{VII}} + \underbrace{\sum_{m=i}^{k-1} p'_m}_{\text{VIII}} + \underbrace{\sum_{m=k+1}^{j} p'_m}_{\text{IX}}$$

$$\boldsymbol{c(i,j) = c(i, k\text{-}1) + c(k,j) + \mathrm{P}(i,j)}$$

Árvore de Custo Mínimo

O lema leva a recorrência:

$$\begin{cases} c(i,i) = 0 \\ c(i,j) = \min_{i<k\le j}\{c(i,k-1)+c(k,j)\}+P(i,j) \end{cases}$$

Número de pares distintos = $n(n+1)/2 \rightarrow O(n^2)$

Árvore de Custo Mínimo

Algoritmo

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até $n-d$ faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j-1] + f_j + f'_j$
    $c[i,j] \leftarrow \min_{i < k \leq j} \{ c[i,k-1] + c[k,j] \} + F[i,j]$

Árvore de Custo Mínimo

Algoritmo

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até $n-d$ faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j-1] + f_j + f'_j$
    $c[i,j] \leftarrow \min_{i < k \leq j} \{ c[i,k-1] + c[k,j] \} + F[i,j]$

Complexidade? $\Theta(n^3)$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

$$F = \begin{vmatrix} 2 & & & & \\ - & 1 & & & \\ - & - & 1 & & \\ - & - & - & 1 & \\ - & - & - & - & 1 \end{vmatrix} \quad c = \begin{vmatrix} 0 & & & & \\ - & 0 & & & \\ - & - & 0 & & \\ - & - & - & 0 & \\ - & - & - & - & 0 \end{vmatrix} \quad k = \begin{vmatrix} - & & & & \\ - & - & & & \\ - & - & - & & \\ - & - & - & - & \\ - & - & - & - & - \end{vmatrix}$$

$$F(i,i) = f'_j \qquad c(i,i) = 0 \qquad k(i,i) = -$$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
          $i < k \leq j$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

d=1 → 1 nó na árvore
i = 0
j = 1



$T(0,1)$ $F[0,1] = 2 + 10 + 1 = 13$
$k = 1$
$c[0,1] = c[0,0] + c[1,1] + F[0,1] = 13$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j]\} + F[i,j]$
      $i < k \leq j$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & & & \\ - & 1 & 3 & & \\ - & - & 1 & & \\ - & - & - & 1 & \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & & & \\ - & 0 & 3 & & \\ - & - & 0 & & \\ - & - & - & 0 & \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & & & \\ - & - & 2 & & \\ - & - & - & & \\ - & - & - & - & \\ - & - & - & - & - \end{vmatrix}$$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

d=1 → 1 nó na árvore
i = 0
j = 1



$T(0,1)$
- $F[0,1] = 2 + 10 + 1 = 13$
- $k = 1$
- $c[0,1] = c[0,0] + c[1,1] + F[0,1] = 13$

d=1 → 1 nó na árvore
i = 1
j = 2



$T(1,2)$
- $F[1,2] = 1 + 1 + 1 = 3$
- $k = 2$
- $c[1,2] = c[1,1] + c[2,2] + F[1,2] = 3$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & & & \\ - & 1 & 3 & & \\ - & - & 1 & & \\ - & - & - & 1 & \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & & & \\ - & 0 & 3 & & \\ - & - & 0 & & \\ - & - & - & 0 & \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & & & \\ - & - & 2 & & \\ - & - & - & & \\ - & - & - & - & \\ - & - & - & - & - \end{vmatrix}$$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
      $i < k \leq j$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

d=1 → 1 nó na árvore
i = 2
j = 3



$T(2,3)$
- $F[2,3] = 1 + 3 + 1 = 5$
- $k = 3$
- $c[2,3] = c[2,2] + c[3,3] + F[2,3] = 5$

d=1 → 1 nó na árvore
i = 3
j = 4



$T(3,4)$
- $F[3,4] = 1 + 2 + 1 = 4$
- $k = 4$
- $c[3,4] = c[3,3] + c[4,4] + F[3,4] = 4$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & & & \\ - & 1 & 3 & & \\ - & - & 1 & 5 & \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad \begin{vmatrix} 0 & 13 & & & \\ - & 0 & 3 & & \\ - & - & 0 & 5 & \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad \begin{vmatrix} - & 1 & & & \\ - & - & 2 & & \\ - & - & - & 3 & \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
    $i < k \leq j$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

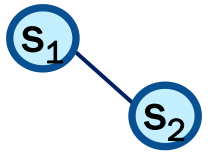
$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

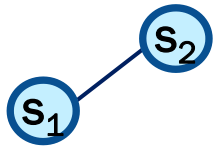
d=2 → 2 nós na árvore

i = 0

j = 2



$T(0,2)$
- $F[0,2] = 13 + 1 + 1 = 15$
- $k = 1$
- $c[0,2] = c[0,0] + c[1,2] + F[0,2]$
  $= 0 + 3 + 15 = 18$



$T(0,2)$
- $k = 2$
- $c[0,2] = c[0,1] + c[2,2] + F[1,2]$
  $= 13 + 0 + 15 = 28$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & 15 & & \\ - & 1 & 3 & & \\ - & - & 1 & 5 & \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & 18 & & \\ - & 0 & 3 & & \\ - & - & 0 & 5 & \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & 1 & & \\ - & - & 2 & & \\ - & - & - & 3 & \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
      $i < k \leq j$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

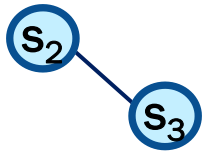
$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

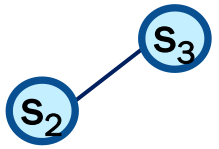
d=2 → 2 nós na árvore

i = 1

j = 3



$T(1,3)$
- $F[1,3] = 3 + 3 + 1 = 7$
- $k = 2$
- $c[1,3] = c[1,1] + c[2,3] + F[1,3]$
  $= 0 + 5 + 7 = 12$



$T(1,3)$
- $k = 3$
- $c[1,3] = c[1,2] + c[3,3] + F[1,3]$
  $= 3 + 0 + 7 = 10$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & 15 & & \\ - & 1 & 3 & 7 & \\ - & - & 1 & 5 & \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & 18 & & \\ - & 0 & 3 & 10 & \\ - & - & 0 & 5 & \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & 1 & & \\ - & - & 2 & 3 & \\ - & - & - & 3 & \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
    $i < k \leq j$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

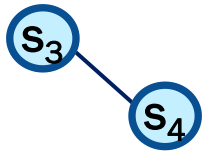
$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

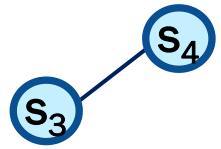
d=2 → 2 nós na árvore

i = 2

j = 4



$T(2,4)$ ⎧ $F[2,4] = 1 + 2 + 1 = 4$
⎪ $k = 3$
⎩ $c[2,4] = c[2,2] + c[3,4] + F[2,4]$
$= 0 + 4 + 4 = 8$



$T(2,4)$ ⎧ $k = 4$
⎩ $c[2,4] = c[2,3] + c[4,4] + F[2,4]$
$= 5 + 0 + 4 = 9$

2	13	15		
-	1	3	7	
-	-	1	5	4
-	-	-	1	4
-	-	-	-	1

0	13	18		
-	0	3	10	
-	-	0	5	8
-	-	-	0	4
-	-	-	-	0

-	1	1		
-	-	2	3	
-	-	-	3	3
-	-	-	-	4
-	-	-	-	-

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
      $i < k \leq j$

Exemplo:

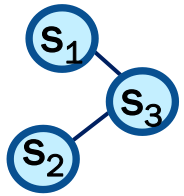
j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

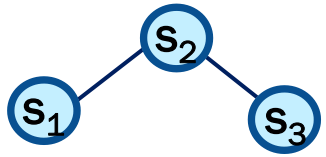
$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

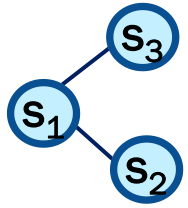
d=3 → 3 nós na árvore
i = 0
j = 3



$T(0,3)$
- $F[0,3] = 15 + 3 + 1 = 19$
- $k = 1$
- $c[0,3] = c[0,0] + c[1,3] + F[0,3]$
  $= 0 + 10 + 19 = 29$



$k = 2$
$c[0,3] = c[0,1] + c[2,3] + F[0,3]$
$= 13 + 5 + 19 = 37$



$k = 3$
$c[0,3] = c[0,2] + c[3,3] + F[0,3]$
$= 18 + 0 + 19 = 37$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & 15 & 19 & \\ - & 1 & 3 & 7 & \\ - & - & 1 & 5 & 4 \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & 18 & 29 & \\ - & 0 & 3 & 10 & \\ - & - & 0 & 5 & 8 \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & 1 & 1 & \\ - & - & 2 & 3 & \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até n-d faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j] \} + F[i,j]$
      $i < k \leq j$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

T(i,j) ⟶ a árvore de busca ótima para as chaves $\{s_{i+1}, ..., s_j\}$, $i \leq j$.

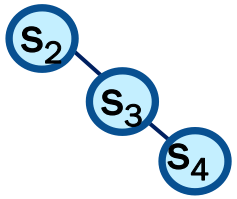
$c(i,j)$ ⟶ custo da árvore T(i,j)

F(i,j) ⟶ a soma das probabilidades relativas as chaves $\{s_{i+1}, ..., s_j\}$

d=3 → 3 nós na árvore

i = 1

j = 4



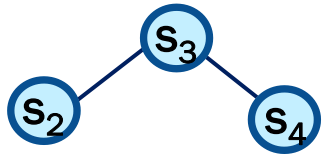
$T(1,4)$

$F[1,4] = 7 + 2 + 1 = 10$

$k = 2$

$c[1,4] = c[1,1] + c[2,4] + F[1,4]$

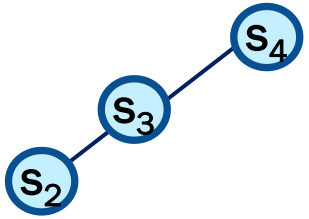
$= 0 + 8 + 10 = 18$



$k = 3$

$c[1,4] = c[1,2] + c[3,4] + F1,4]$

$= 3 + 4 + 10 = 17$



$k = 4$

$c[1,4] = c[1,3] + c[4,4] + F[1,4]$

$= 10 + 0 + 10 = 20$

$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & 15 & 19 & \\ - & 1 & 3 & 7 & 10 \\ - & - & 1 & 5 & 4 \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & 18 & 29 & \\ - & 0 & 3 & 10 & 17 \\ - & - & 0 & 5 & 8 \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & 1 & 1 & \\ - & - & 2 & 3 & 3 \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Para $j \leftarrow 0$ até n faça

  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça

  para $i \leftarrow 0$ até n-d faça

    $j \leftarrow i + d$

    $F[i,j] \leftarrow F[i,j\text{-}1] + f_j + f'_j$

    $c[i,j] \leftarrow \min \{ c[i,k\text{-}1] + c[k,j]\} + F[i,j]$

    $i < k \leq j$

Exemplo:

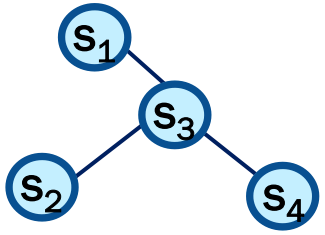
j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

$$F = \begin{vmatrix} 2 & 13 & 15 & 19 & \\ - & 1 & 3 & 7 & 10 \\ - & - & 1 & 5 & 4 \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad c = \begin{vmatrix} 0 & 13 & 18 & 29 & \\ - & 0 & 3 & 10 & 17 \\ - & - & 0 & 5 & 8 \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad k = \begin{vmatrix} - & 1 & 1 & 1 & \\ - & - & 2 & 3 & 3 \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

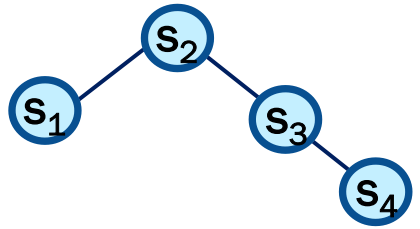
Para $j \leftarrow 0$ até n faça
  $c[j,j] \leftarrow 0$; $F[j,j] \leftarrow f'_j$

Para $d \leftarrow 1$ até n faça
  para $i \leftarrow 0$ até $n-d$ faça
    $j \leftarrow i + d$
    $F[i,j] \leftarrow F[i,j-1] + f_j + f'_j$
    $c[i,j] \leftarrow \min \{ c[i,k-1] + c[k,j] \} + F[i,j]$
      $i < k \leq j$

d=4 → 4 nós na árvore
i = 0
j = 4
$T(0,4)$
$F[0,4] = 19 + 2 + 1 = 22$
$k = 1$
$c[0,4] = c[0,0] + c[1,4] + F[0,4]$
$= 0 + 17 + 22 = 39$

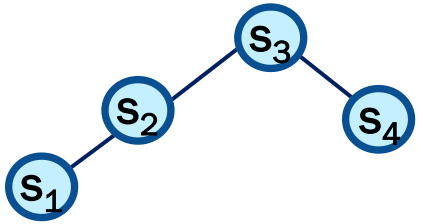


$k = 2$



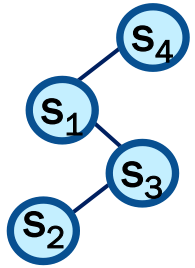
$c[0,4] = c[0,1] + c[2,4] + F[0,4]$
$= 13 + 8 + 22 = 43$

$k = 3$



$c[0,4] = c[0,2] + c[3,4] + F[0,4]$
$= 18 + 4 + 22 = 44$

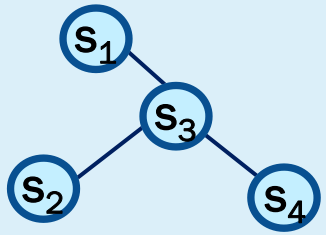
$k = 4$



$c[0,4] = c[0,3] + c[4,4] + F[0,4]$
$= 29 + 0 + 22 = 51$

Exemplo:

j	0	1	2	3	4
f_j	-	10	1	3	2
f'_j	2	1	1	1	1

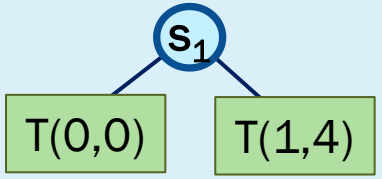


$$\boldsymbol{F} = \begin{vmatrix} 2 & 13 & 15 & 19 & 22 \\ - & 1 & 3 & 7 & 10 \\ - & - & 1 & 5 & 4 \\ - & - & - & 1 & 4 \\ - & - & - & - & 1 \end{vmatrix} \quad \boldsymbol{c} = \begin{vmatrix} 0 & 13 & 18 & 29 & 39 \\ - & 0 & 3 & 10 & 17 \\ - & - & 0 & 5 & 8 \\ - & - & - & 0 & 4 \\ - & - & - & - & 0 \end{vmatrix} \quad \boldsymbol{k} = \begin{vmatrix} - & 1 & 1 & 1 & 1 \\ - & - & 2 & 3 & 3 \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Exemplo: Recuperando a árvore a partir da matriz *k*

$$k = \begin{vmatrix} - & 1 & 1 & 1 & 1 \\ - & - & 2 & 3 & 3 \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Raiz da árvore T(0,4) = s_1

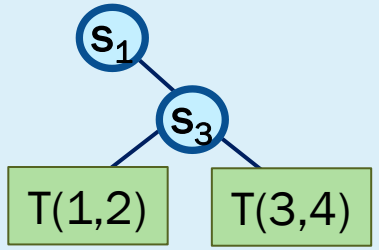


Exemplo: Recuperando a árvore a partir da matriz *k*

$$k = \begin{vmatrix} - & 1 & 1 & 1 & 1 \\ - & - & 2 & 3 & 3 \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Raiz da árvore T(0,0) = -

Raiz da árvore T(1,4) = s_3

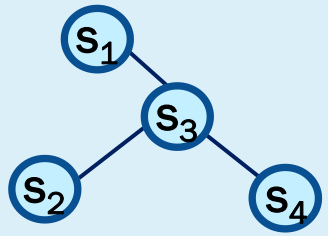


Exemplo: Recuperando a árvore a partir da matriz *k*

$$k = \begin{vmatrix} - & 1 & 1 & 1 & 1 \\ - & - & 2 & 3 & 3 \\ - & - & - & 3 & 3 \\ - & - & - & - & 4 \\ - & - & - & - & - \end{vmatrix}$$

Raiz da árvore T(1,2) = s_2

Raiz da árvore T(3,4) = s_4



Problema do Troco

Dado um sistema monetário em que há disponibilidade infinita de recursos, descreva um algoritmo para pagar um certo valor ao cliente com o menor número de moedas possível.



Problema do Troco

- *Sistema no Brasil:*

Notas de R$100, 50, 20, 10, 5 e 2

Moedas de R$ 1, 0,50, 0,25, 0,10, 0,05 e 0,01.



Suponha que você quer dar um troco de R$ 48,25.

Como você faria?

Problema do Troco

Você pensou em

2 notas de R$20 + 1 nota de R$5 + 1 nota (moeda) de R$2 + 1 moeda de R$1 + 1 moeda de R$0,25 ?

Se pensou, você embutiu o problema de dar o menor número de notas possível e usou uma estratégia gulosa para resolver.

Seu cérebro agiu rápido...



Problema do Troco

A estratégia gulosa sempre funciona?



Problema do Troco

A estratégia gulosa sempre funciona?

Nem sempre!



Problema do Troco



Dado um sistema monetário em que há disponibilidade infinita de recursos, descreva um algoritmo para pagar um certo valor ao cliente com o menor número de moedas possível.

Algoritmo guloso é muito eficiente mas sua aplicabilidade se restringe a um número limitado de casos de teste. (Exemplo - Moedas: 1, 4, 6; Troco: 8).

Algoritmo de Programação Dinâmica consegue fornecer a solução ótima ou informar que o valor não pode ser pago com o sistema monetário adotado.

Problema do Troco

- Tabela de informações:
  - *C[1...n][0...N]*
  - *Linhas: descrições e valores válidos de moeda.*
  - *Colunas: valores de 0 até o valor N que deve ser pago ao cliente.*
  - *C[i][j] = menor número de moedas com descrição até i que pode ser usado para pagar a quantia j.*
  - *Solução: C[n][N]*

	0	1	2
1: 1			
2: 4			

Problema do Troco

- Como combinar subproblemas?

  Ao calcular C[i,j], temos 2 opções

  - *Não incluir nenhuma moeda de denominação i*
    - C[i][j] = C[i-1][j]
  - *Incluir pelo menos uma moeda de denominação i*
    - C[i][j] = C[i][j-valor(i)]+1

$$C[i][j] = \min (c[i-1][j], 1 + c[i][j-valor(i)])$$

Obs.: Elementos fora da tabela têm valor $+\infty$.

Se, ao final, $C[n,N] = +\infty$, então o valor não pode ser pago.

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
            c[i][j] ← se i = 1 e j < valor [i] então +∞
                      senão se i = 1 então 1 + c[1][j-valor[1]]
                      senão se j < valor [i] então c[i-1][j]
                      senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
      para j ← 1 até N faça
         c[i][j] ← se i = 1 e j < valor [i] então +∞
                   senão se i = 1 então 1 + c[1][j-valor[1]]
                   senão se j < valor [i] então c[i-1][j]
                   senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```

	0	1	2	3	4	5	6	7	8
1: 1	0								
2: 4	0								
3: 6	0								

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=1
j= 1,...N

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0								
3: 6	0								

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=2
j=1
Valor[i] = 4

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1							
3: 6	0								

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=2
j=2,3
Valor[i] = 4

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3					
3: 6	0								

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```


i=2
j=4
Valor[i] = 4
c[i-1][j] = c[1][4] = 4
C[i][j-valor[i]] = c[2][0]=0

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1				
3: 6	0								

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=2
j=5,6,7,8
Valor[i] = 4

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0								

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=3
j=1,2,3,4,5
Valor[i] = 6

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0	1	2	3	1	2			

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=3
j=6
Valor[i] = 6
c[i-1][j] = c[2][6] = 3
C[i][j-valor[i]] = c[3][0]=0

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0	1	2	3	1	2	1		

Problema do Troco

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```



i=3
j=7,8
Valor[i] = 6

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0	1	2	3	1	2	1	2	2

Problema do Troco

Recuperando as notas a partir da tabela

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min ( c[i-1][j], 1 + c[i][j-valor[i]])
```

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0	1	2	3	1	2	1	2	2

0 moedas de valor 6

Problema do Troco

Recuperando as notas a partir da tabela

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0	1	2	3	1	2	1	2	2

1 moeda de valor 4

Problema do Troco

Recuperando as notas a partir da tabela

Função Troco (*N*)

```
vetor valor [1..n] = [1, 4, 6]
vetor c[1..n][0..N]

para i ← 1 até n faça c[i][0] ← 0

para i ← 1 até n faça
        para j ← 1 até N faça
           c[i][j] ← se i = 1 e j < valor [i] então +∞
                     senão se i = 1 então 1 + c[1][j-valor[1]]
                     senão se j < valor [i] então c[i-1][j]
                     senão min (c[i-1][j], 1 + c[i][j-valor[i]])
```

	0	1	2	3	4	5	6	7	8
1: 1	0	1	2	3	4	5	6	7	8
2: 4	0	1	2	3	1	2	3	4	2
3: 6	0	1	2	3	1	2	1	2	2

2 moedas de valor 4

Maior Subsequência Comum

Maior Subsequência Comum (LCS)

X = { A-E-F-G-H-A-B-C}

Y = { B-E-A-G-H-A-E-C-E}

X = { A-E-F-G-H-A-B-C}

Y = { B-E-A-G-H-A-E-C-E}

LCS = { A-G-H-A-C}

Maior Subsequência Comum (LCS)

- **Comparar cada subsequência de X com Y**
- **Se $|X|=n$ e $|Y|=m$ existem 2^n subsequências de X**
- **Solução é $O(m\ 2^n)$**
- **Todavia subsequências menores fazem parte de sequências maiores**

Maior Subsequência Comum (LCS)

Programação Dinâmica

Determinar LCS para prefixos de X e Y: LCS (X,Y)

X_i, Y_j são prefixos de X e Y de tamanhos i e j

c[i, j] é o tamanho da LCS de X_i e Y_j

c[n,m] guarda a LCS de X e Y

Maior Subsequência Comum (LCS)

Solução Recursiva

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \textbf{Se } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \textbf{Caso Contrário} \end{cases}$$

Início: $i = j = 0$ (subsequências vazias)

X_0 e Y_0 são sequências vazias para todo i e j

$$c[0,j] = c[i,0] = 0$$

Maior Subsequência Comum (LCS)

Solução Recursiva

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \textbf{Se } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \textbf{Caso Contrário} \end{cases}$$

Primeiro Caso:

x[i] = y[j]: mais um símbolo em X e Y confere

Assim: LCS para X_i e Y_j é igual ao LCS de X_{i-1} e Y_{i-1} , mais 1

$$LCS(X_i, Y_j) = LCS(X_{i-1}, Y_{j-1}) + 1$$

Maior Subsequência Comum (LCS)

Solução Recursiva

$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \textbf{Se } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \textbf{Caso Contrário} \end{cases}$$

Segundo Caso:

x[i] ≠ y[j]: símbolo não confere

Assim: LCS não pode ser melhorada

$$\text{Max}\left(LCS(X_i, Y_{j-1}) \;\; e \;\; LCS(X_{i-1}, Y_j) \right)$$

Maior Subsequência Comum (LCS)

```
LCS(X, Y)
1. n = Comprimento de (X)
2. m = Comprimento de (Y)
3. for i = 1 to n      c[i,0] = 0   // caso base: Y0
4. for j = 1 to m      c[0,j] = 0   // caso base: X0
5. for i = 1 to n
6.    for j = 1 to m
7.     if ( xi == yj )
8.           c[i,j] = c[i-1,j-1] + 1
9.     else      c[i,j] = max( c[i-1,j], c[i,j-1] )
10. return c(n, m)
```

Maior Subsequência Comum (LCS)

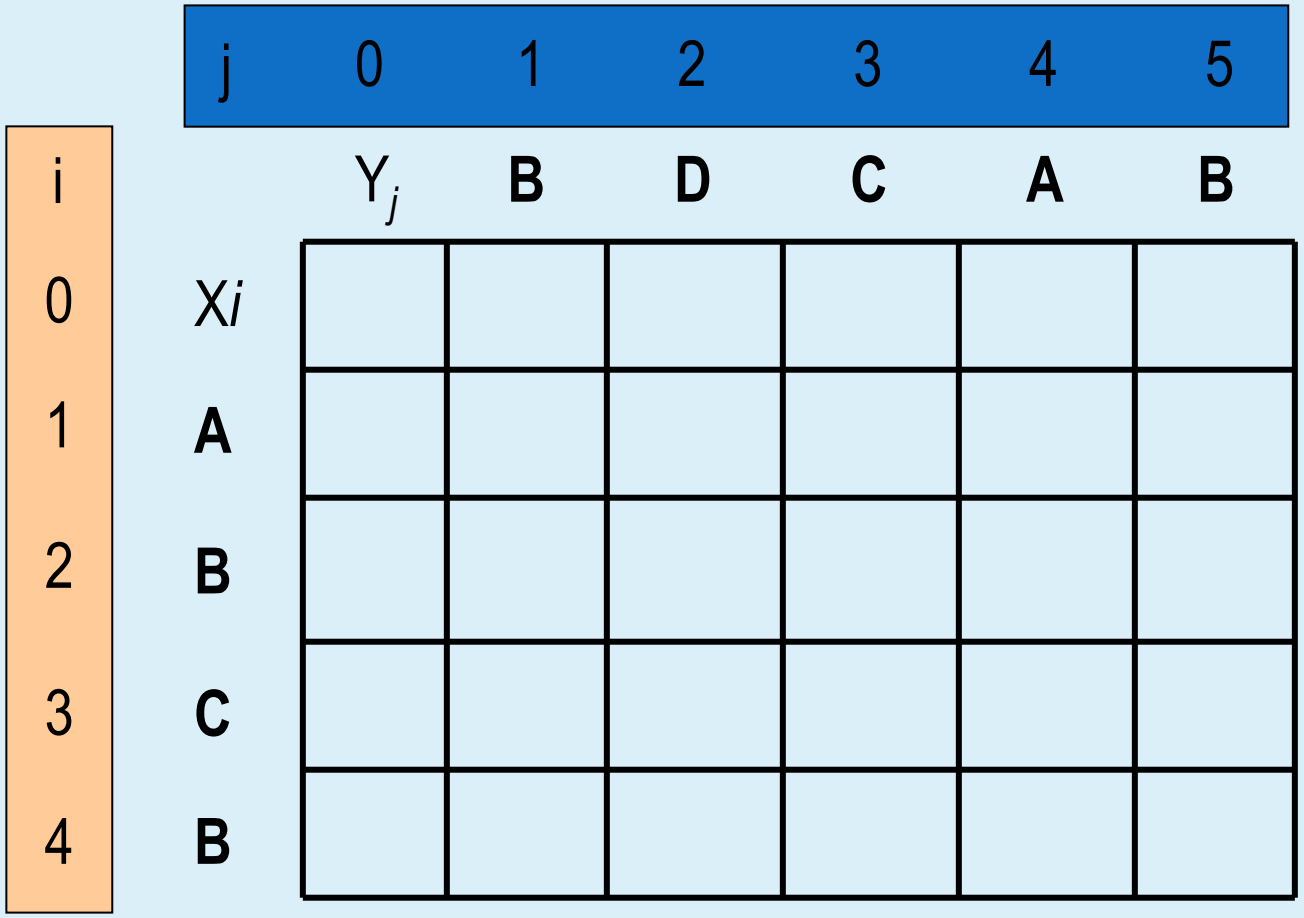
Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

```
for i = 1 to n    c[i,0] = 0
for j = 1 to m    c[0,j] = 0
```

i	j	0	1	2	3	4	5
		Y_j	B	D	C	A	B
0	X_i	0	0	0	0	0	0
1	A	0					
2	B	0					
3	C	0					
4	B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

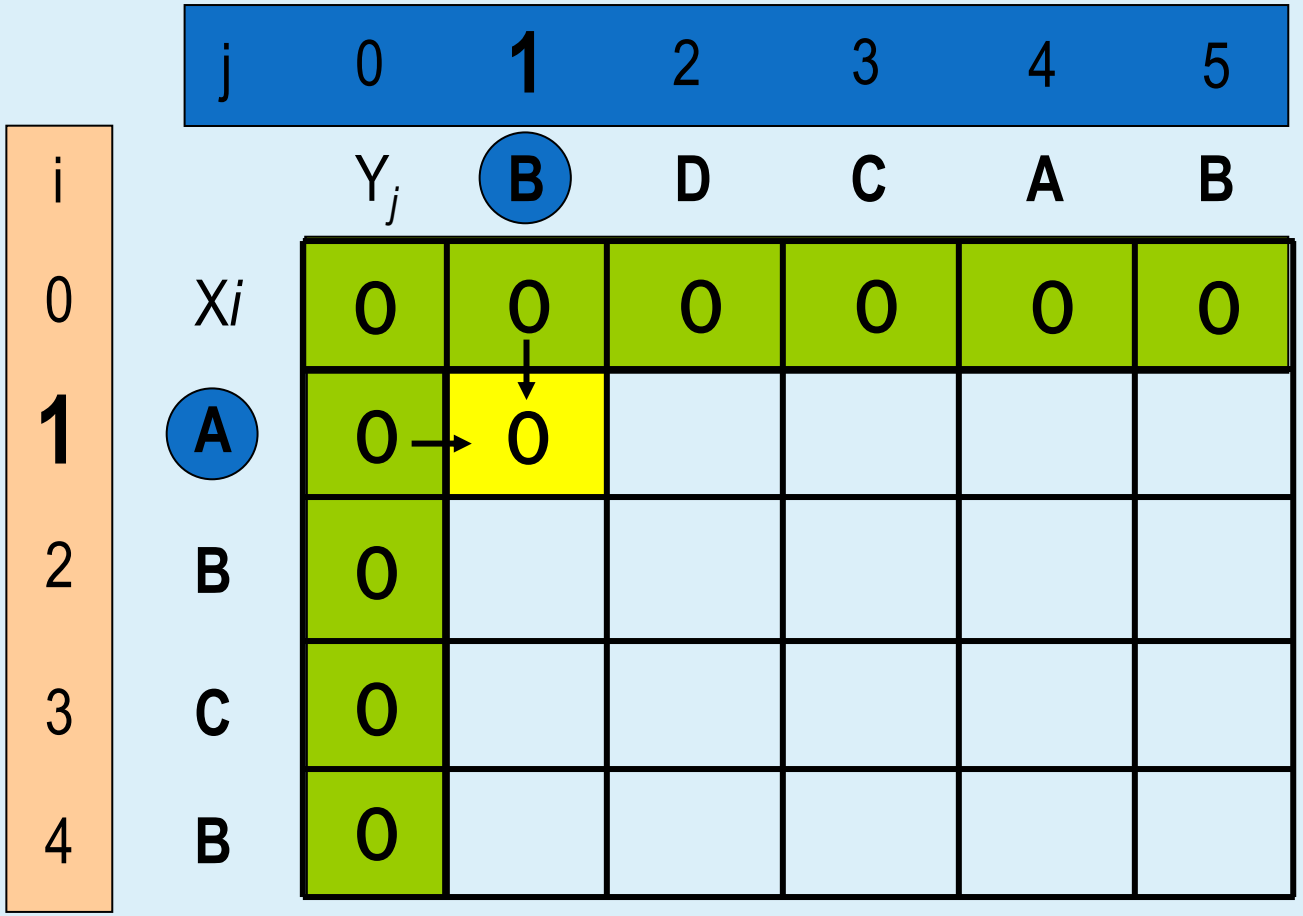
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i	j	0	1	2	3	4	5
		Y_j	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0 →	0			
2	B	0					
3	C	0					
4	B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

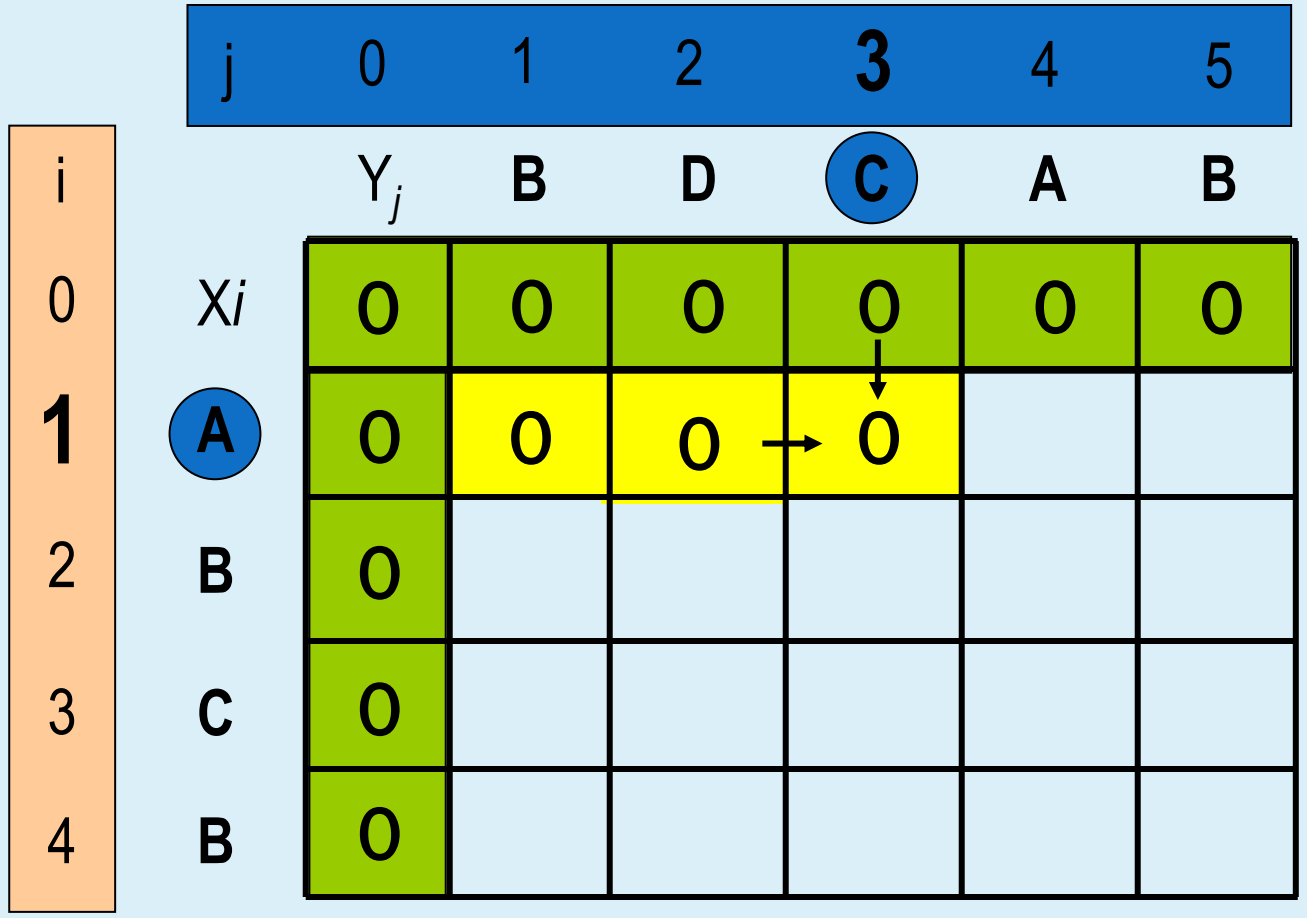
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

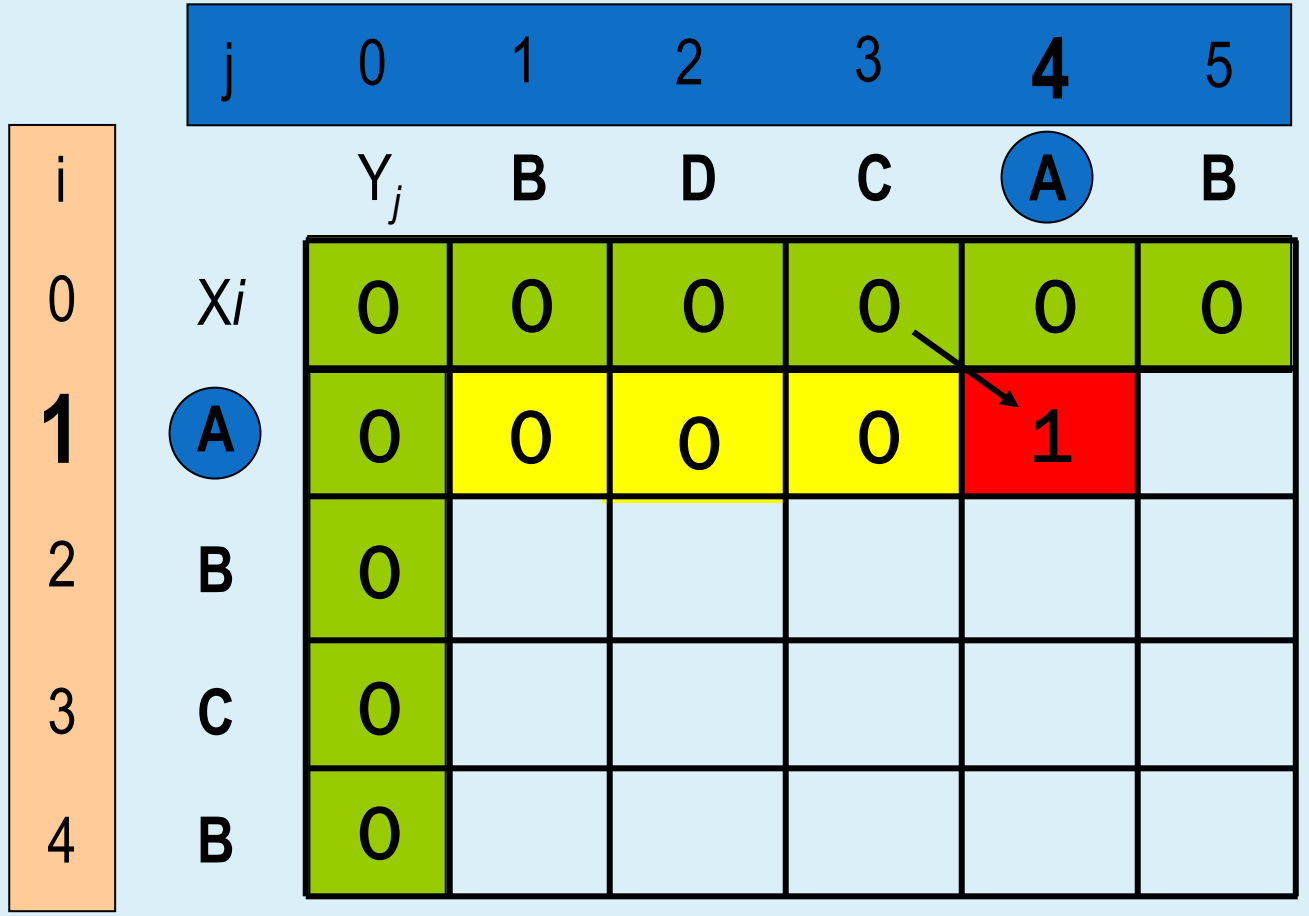
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i
0
1
2
3
4

j	0	1	2	3	4	5
	Y_j	B	D	C	A	B
Xi	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0					
C	0					
B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

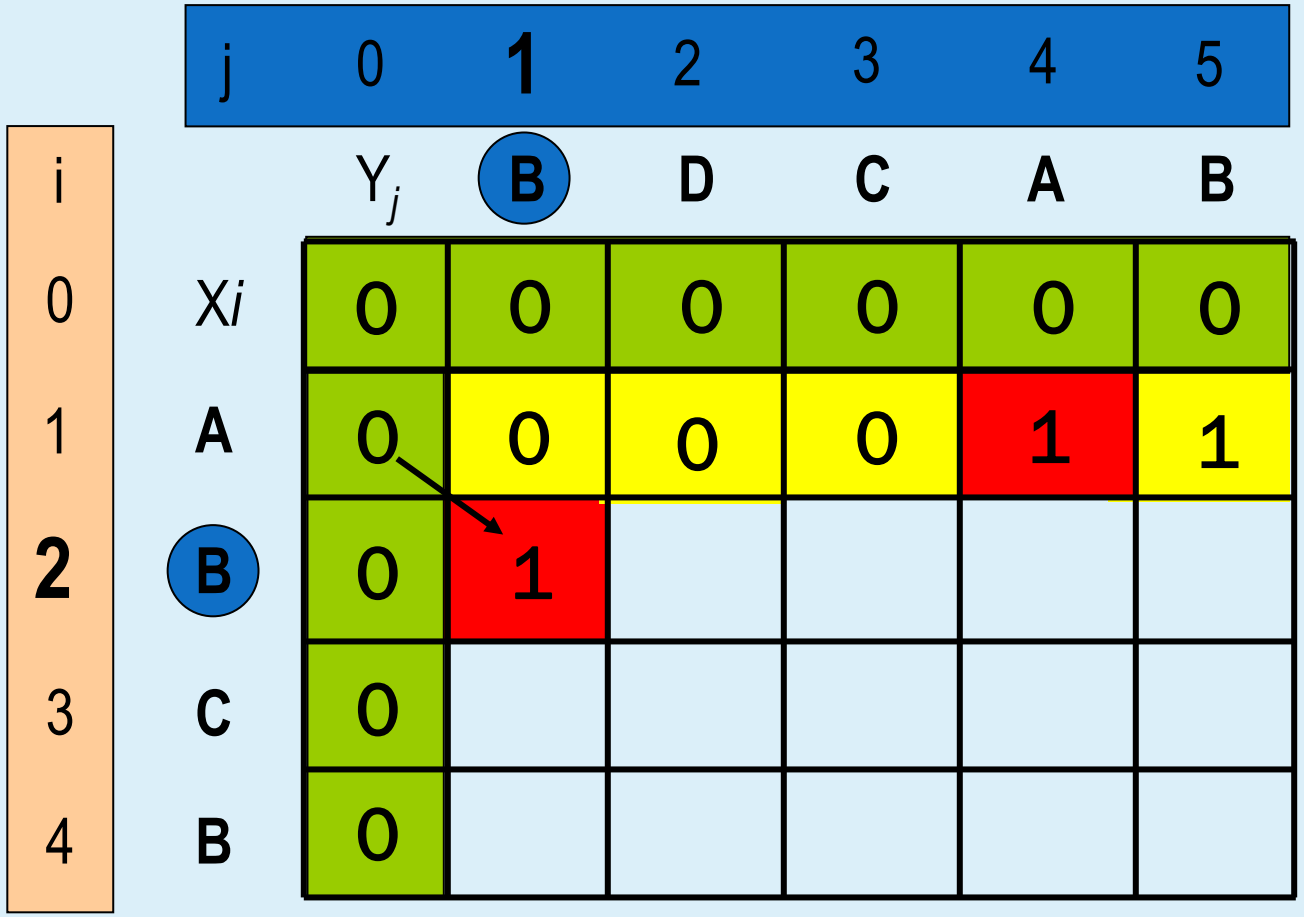
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i	j	0	1	**2**	3	4	5
		Y_j	B	**D**	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	→ 1			
3	C	0					
4	B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i	j	0	1	2	3	4	5
		Y_j	B	D	C	A	B
0	X_i	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1 →	1		
3	C	0					
4	B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

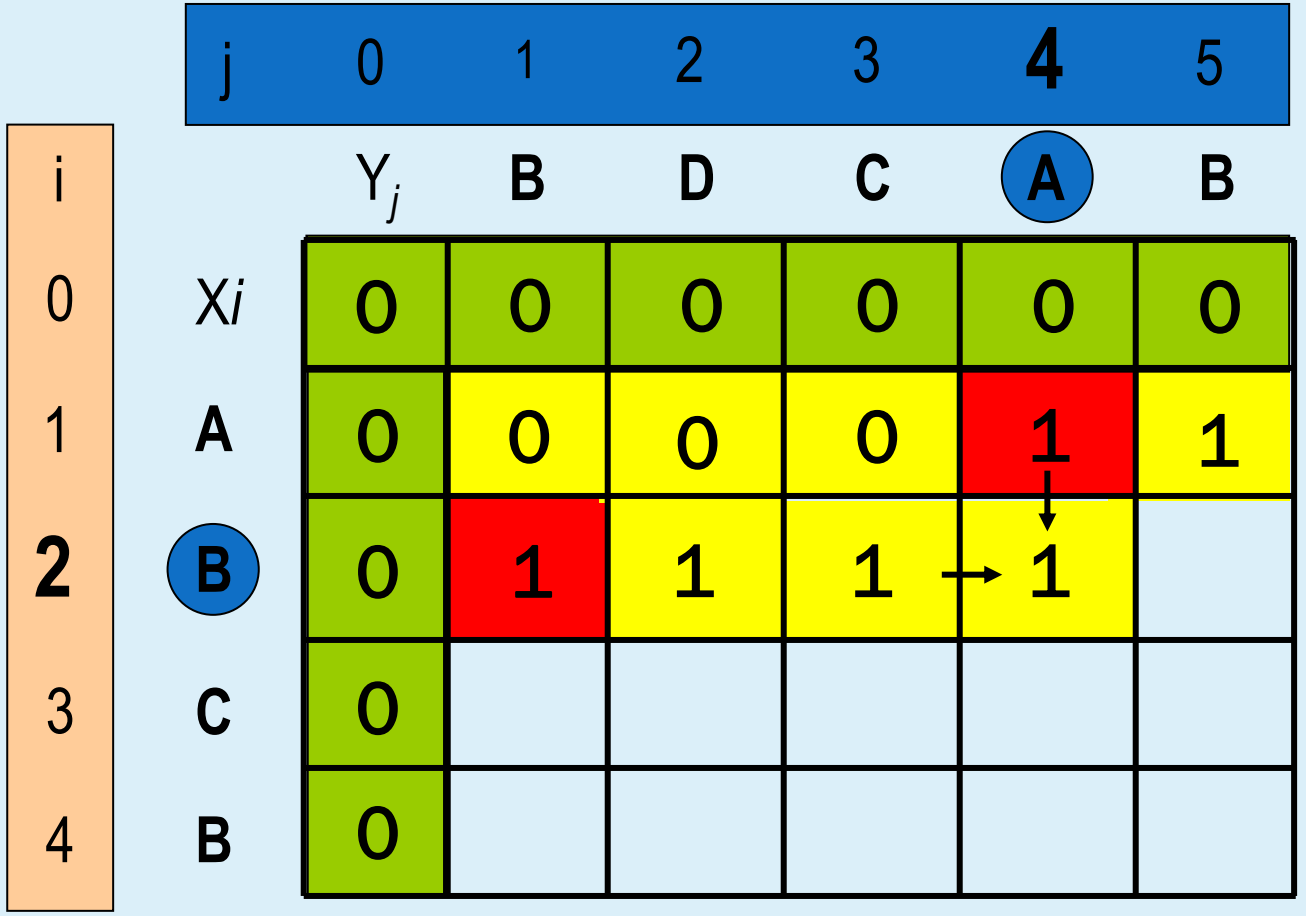
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i		j	0	1	2	3	4	5
		Y_j	B	D	C	A	B
0	X_i		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0					
4	B		0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i		j	0	1	2	3	4	5
				Y_j	B	D	C	A	B
0		X_i	0	0	0	0	0	0
1		A	0	0	0	0	1	1
2		B	0	1	1	1	1	2
3		C	0	1				
4		B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i		j	0	1	2	3	4	5
			Y_j	B	D	C	A	B
0		X_i	0	0	0	0	0	0
1		A	0	0	0	0	1	1
2		B	0	1	1	1	1	2
3		C	0	1	1			
4		B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

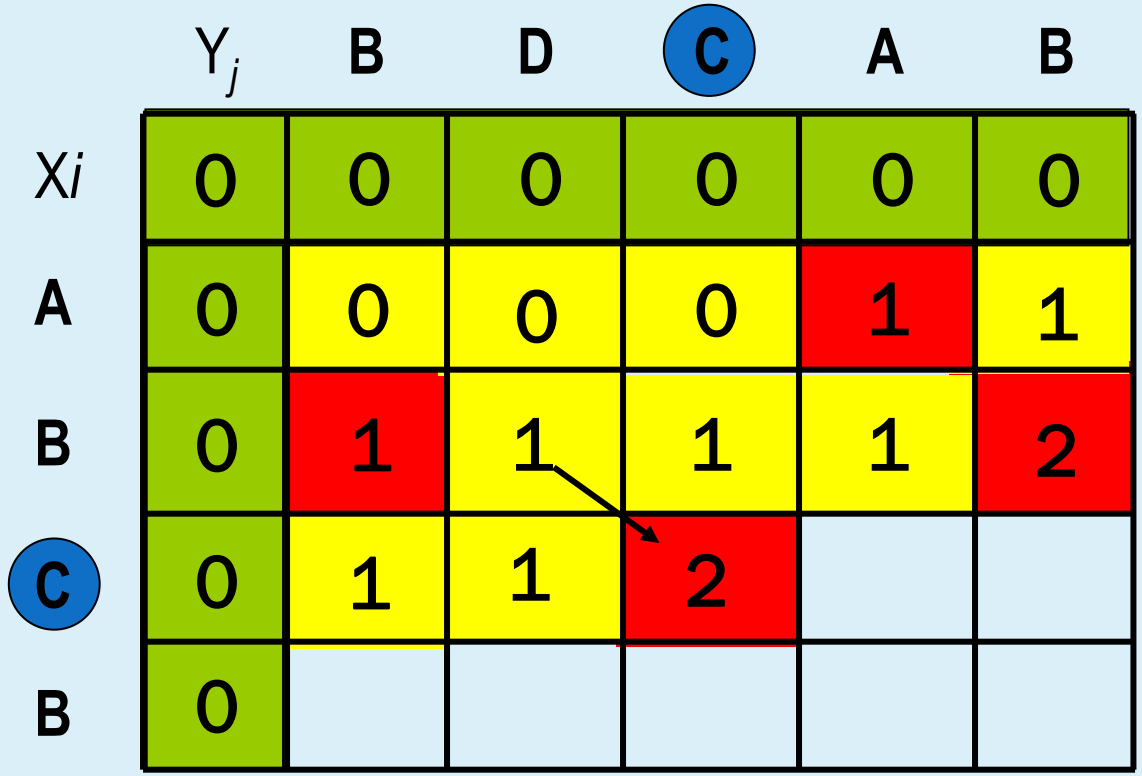
n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i		j	0	1	2	**3**	4	5
			Y_j	**B**	**D**	**C**	**A**	**B**
0			0	0	0	0	0	0
1	**A**		0	0	0	0	1	1
2	**B**		0	1	1	1	1	2
3	**C**		0	1	1	2		
4	**B**		0					



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

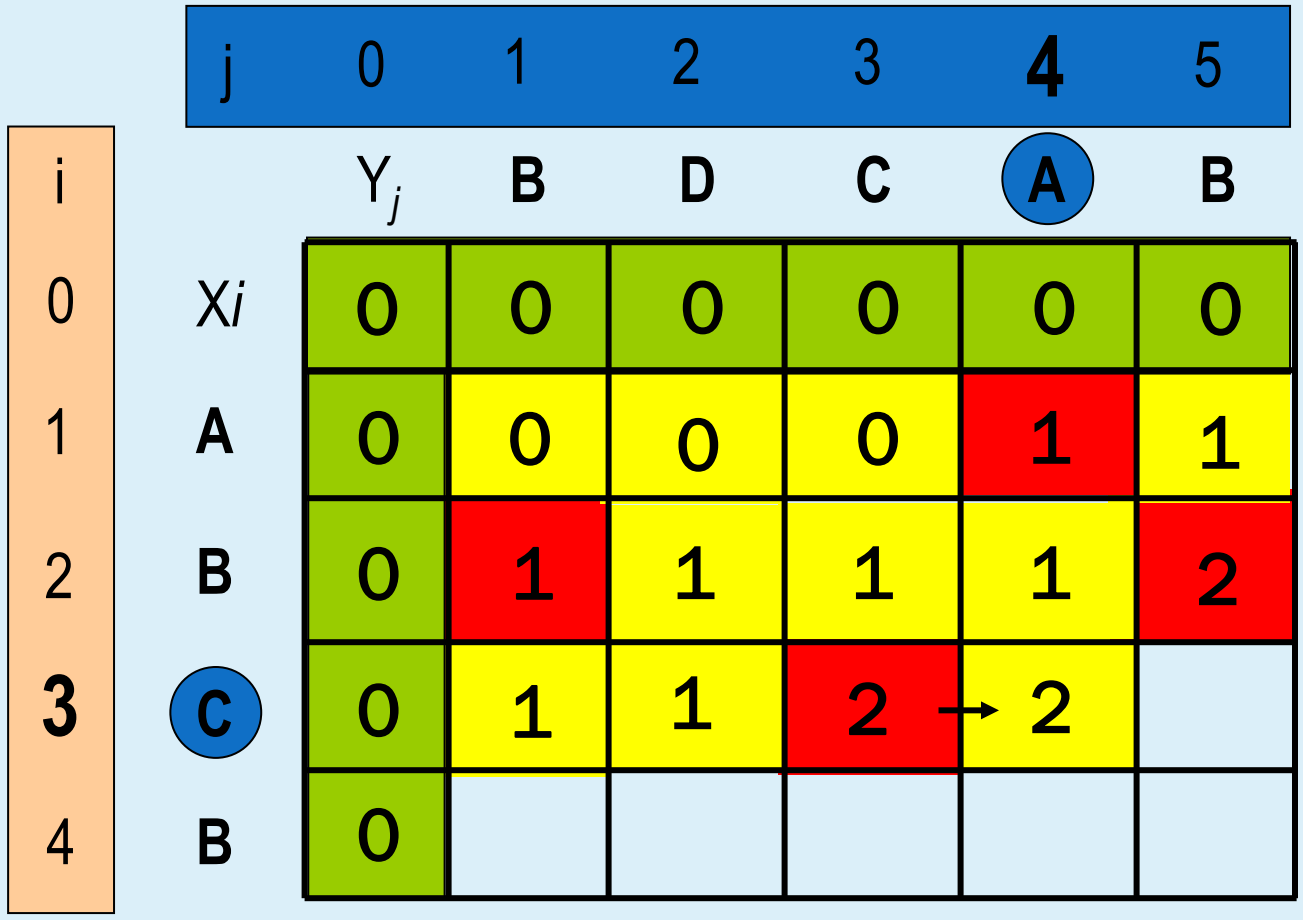
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i	j	0	1	2	3	4	5
		Y_j	B	D	C	A	B
0	X_i	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1	2	2	2
4	B	0					

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i		j	0	1	2	3	4	5
			Y_j	B	D	C	A	B
0	X_i		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1				

Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

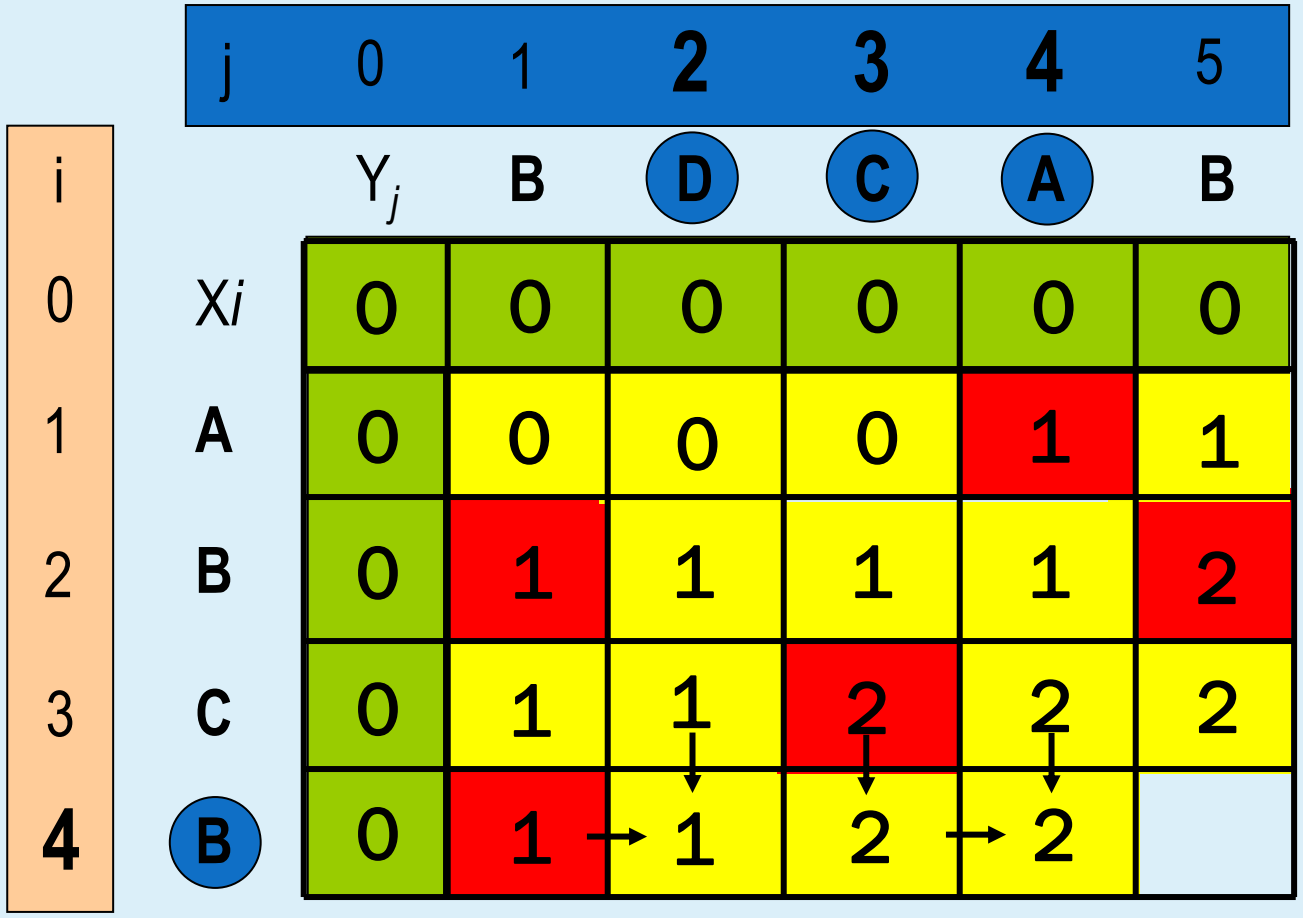
Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```



Maior Subsequência Comum (LCS)

Exemplo

X = ABCB

Y = BDCAB

n = |X| = 4

m = |Y| = 5

ABCB
BDCAB

```
if ( Xi == Yj )
      c[i,j] = c[i-1,j-1] + 1
else
  c[i,j] = max( c[i-1,j], c[i,j-1] )
```

i		j	0	1	2	3	4	5
		Y_j	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	3

Maior Subsequência Comum (LCS)

Recuperação da Sequência

Começar de c[n,m] e retornar

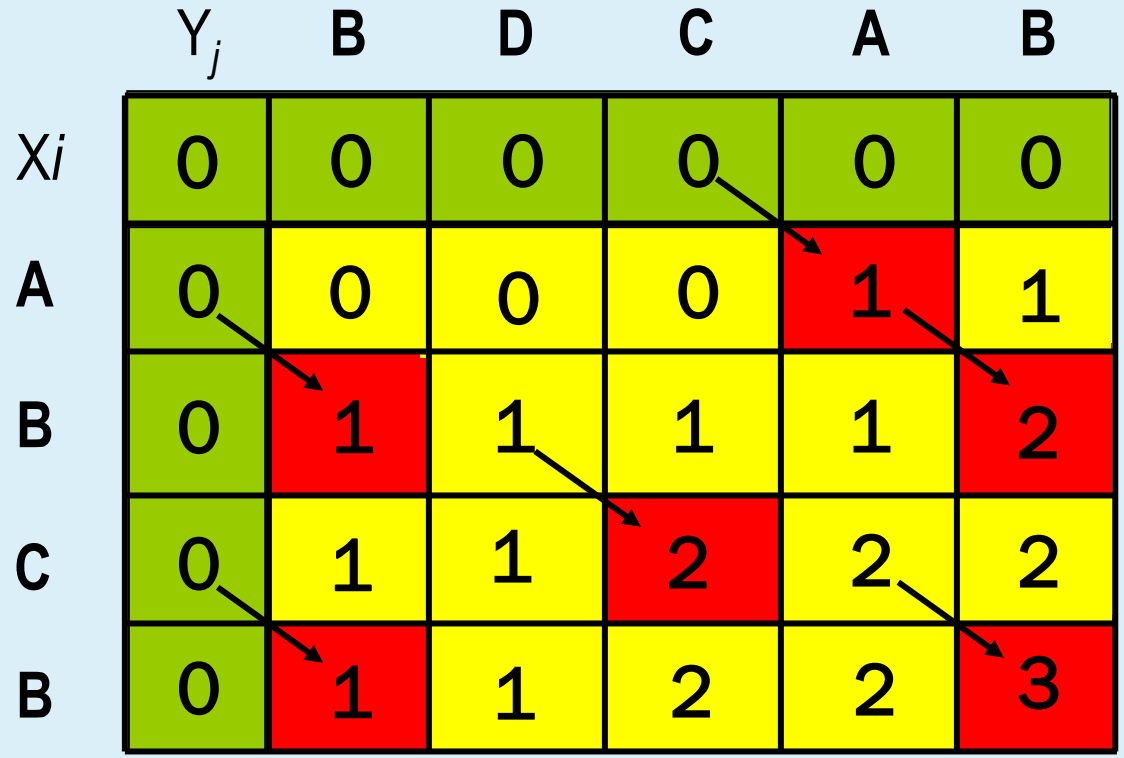
Sempre que c[*i*,*j*] = c[*i*-1, *j*-1]+1, guardar x[*i*] (x[*i*] faz parte da LCS)

Se *i*=0 or *j*=0 (retorno chega ao fim)

A saída é o conjunto de símbolos guardados em ordem inversa

Maior Subsequência Comum (LCS)

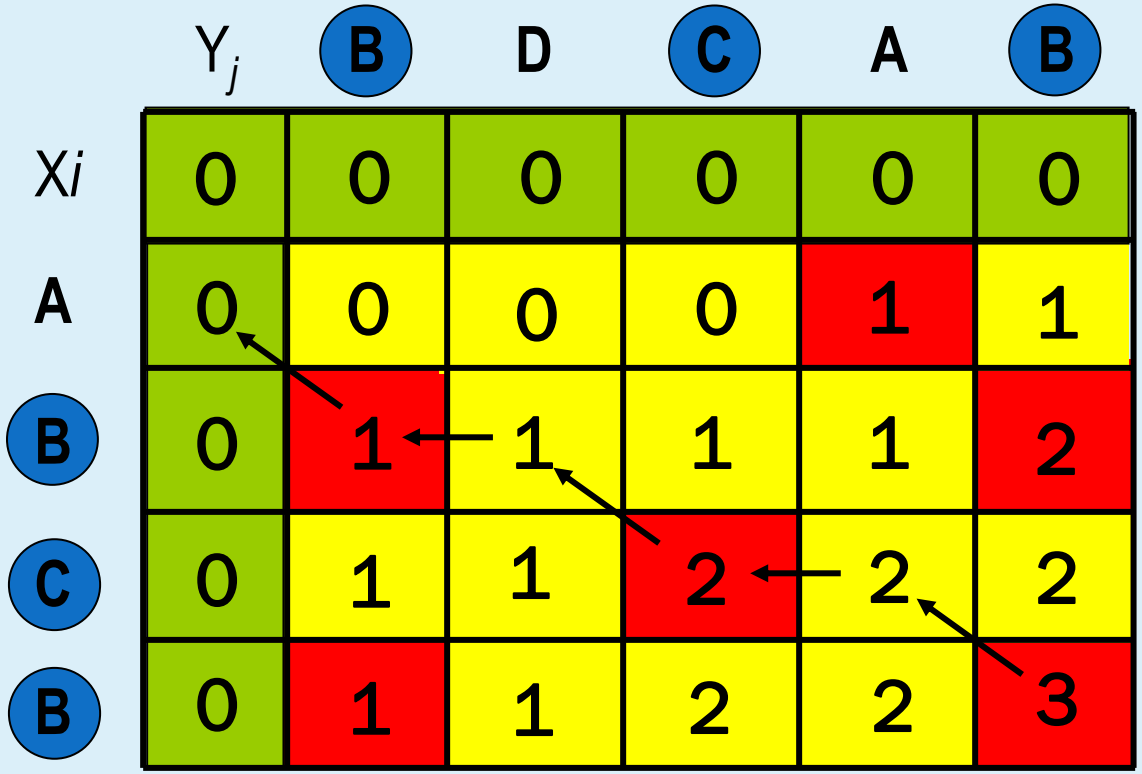
Recuperação da Sequência



$$c[i,j] = c[i\text{-}1, j\text{-}1]+1$$

Maior Subsequência Comum (LCS)

Recuperação da Sequência

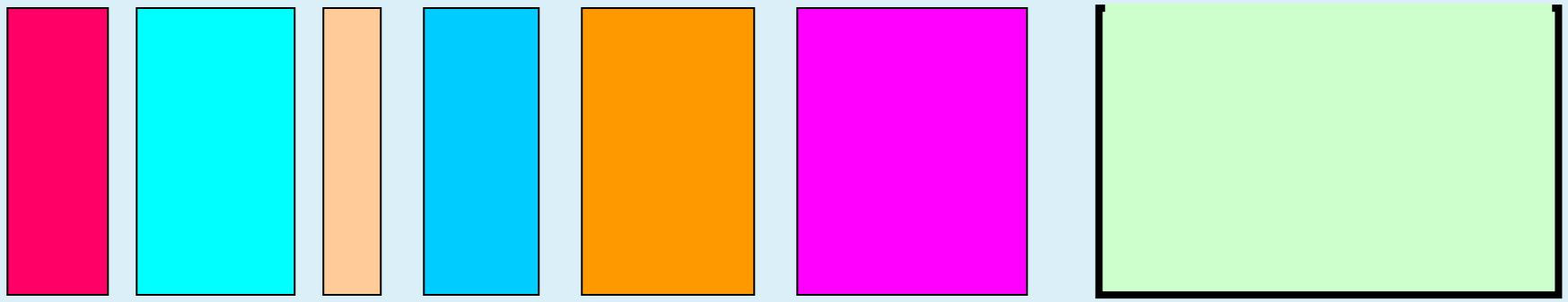


BCB

O Problema da Mochila 0/1

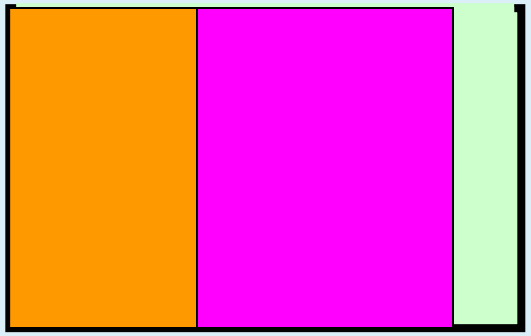
Exercício

O Problema da Mochila 0/1



5 8 7 6 15 10 Mochila

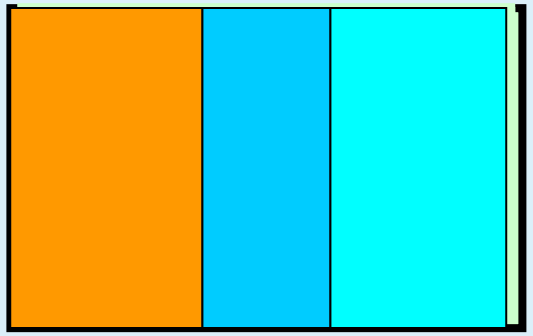
Solução 1



15 10 Valor = 25 (Solução Gulosa)

O Problema da Mochila 0/1

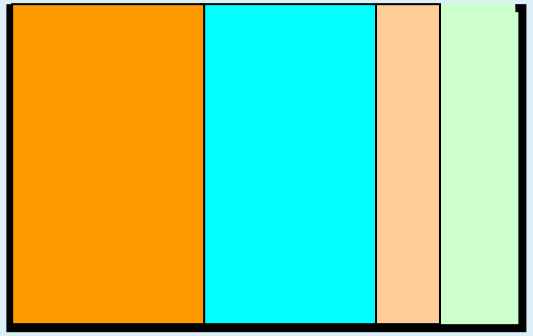
Solução 2



15 6 8

Valor = 29

Solução 3

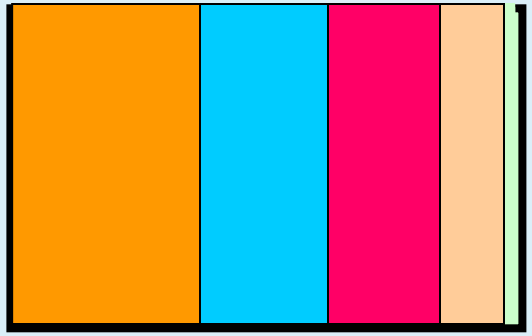


15 8 7

Valor = 30

O Problema da Mochila 0/1

Solução Ótima



Valor = 33

O Problema da Mochila 0/1

$$z = \max \sum_{j=1}^{n} v_j x_j$$

Sujeito a :

$$\sum_{j=1}^{n} w_j x_j \leq W$$

$$v_j, w_j > 0 \text{ e inteiro } j = 1,2,3,..., n$$

$$x_j \in \{0,1\} \; j = 1,2,3,..., n$$

O Problema da Mochila

- Tabela de informações:
  - *V[1...n][0...W]*
  - *Linhas: objetos disponíveis.*
  - *Colunas: valores de 0 até W, peso máximo da mochila.*
  - *V[i][j] = valor máximo dos objetos que podemos transportar se o limite de peso é j.*
  - *Solução: V[n][W]*

	0	1	2
1			
2: 4			

O Problema da Mochila

Como combinar os subproblemas?

O Problema da Mochila

- Ao calcular C[i,j], temos 2 opções
  - *Não incluir objeto i na mochila*
    - $V[i][j] = V[i-1][j]$
  - *Incluir objeto i (peso disponível reduz de w_i e valor aumenta em v_i)*
    - $V[i][j] = V[i-1][j-w_i]+v_i$

$$V[i][j] = \max (V[i-1][j], V[i][j-w_i] + v_i)$$

Obs.: Elementos fora da tabela têm valor $+\infty$. Se, ao final, $C[n,N] = +\infty$, então o valor não pode ser pago.

O Problema da Mochila

W = 8

	0	1	2	3	4	5	6	7	8
w1 =1; v1 = 1	0	1	1	1	1	1	1	1	1
w2 = 2; v2 = 6	0	1	6	7	7	7	7	7	7
w3 = 5; v3 = 18	0	1	6	7	7	18	19	24	25
w4 = 6; v4 = 22	0	1	6	7	7	18	22	24	28
w5 = 7; v5 = 28	0	1	6	7	7	18	22	28	29

O Problema da Mochila

Exercício

1) Dadas as informações anteriores, elabore o algoritmo de programação dinâmica para a mochila 0/1.